

Learner Insight Tracing

Natural Language Knowledge States that Predict Future Work

Anonymous ACL submission

Abstract

Summarizing what a student understands is at the heart of nearly every task in education, yet reliably automating this process has remained a grand challenge. Large language models offer a new opportunity: they can ingest rich records of student work and produce concise natural-language summaries that are interpretable by teachers and transferable across systems. But using these summaries effectively requires a principled way to measure their quality. We introduce Learner Insight Tracing (LIT), a zero-shot framework for generating and evaluating short natural-language insights that describe what a student understands and where they are struggling. We evaluate insights by their ability to predict future student work, using contrastive prediction to compare unstructured future work without relying on exact string matching over code or mathematical reasoning. Our best method, recurrent structured generation, outperforms a broad set of baselines and preserves much of the predictive signal of the full student history while reducing representation size by more than an order of magnitude. We evaluate across 2,240 students on nine assignments spanning three courses, including one open-source dataset to support reproducibility and future comparison. We also deploy LIT in a large online programming course, surfacing insights for 5,449 students to 451 volunteer teachers. We release all prompts and code.

1 Introduction

Understanding students, often called “Noticing” is at the heart of all tasks in education. It refers to a teacher’s ability to understand a student’s work, identify what the student knows, and recognize where they are struggling. Noticing underlies many central tasks in teaching, including grading, deciding when to intervene, selecting the right next piece of content, and understanding how a student is progressing. Authentically automating parts of

this process has long been a grand challenge in education research. Decades of work in student modeling have made significant progress toward this goal, but existing approaches still face several barriers. Real student work is often unstructured, making it difficult to use as model input. Many educational datasets are small, making methods that require large-scale model training impractical. And many systems represent student state using latent vectors or fixed concept estimates, which are difficult for teachers to inspect and hard to transfer across platforms or tasks.

Large language models open a new path through all three of these barriers at once. Because LLMs can process rich, free-form text (i.e. code, written reasoning, error messages, activity logs), they can ingest student work without requiring it to be pre-structured. Because they operate zero-shot, they do not require large labeled datasets for every course. And because their outputs are natural language, they can produce representations that are directly readable by teachers and transferable across platforms without depending on a particular model architecture.

In this work, we present Learner Insight Tracing (LIT), a framework for generating and evaluating short natural-language summaries of student work that we call “Insights.” An Insight is a knowledge state expressed in plain language that summarizes what a student appears to understand, where they are struggling, and what misconceptions are likely to affect their future work. We generate Insights with LLMs and evaluate them using a zero-shot contrastive procedure, asking whether an Insight identifies a student’s future conceptual trajectory among plausible alternatives from other students working on the same task. This evaluation method requires no fine-tuning and no labeled data.

Main Contributions:

1. Predictive evaluation for Learner Insights: We introduce a framework for evaluating knowl-

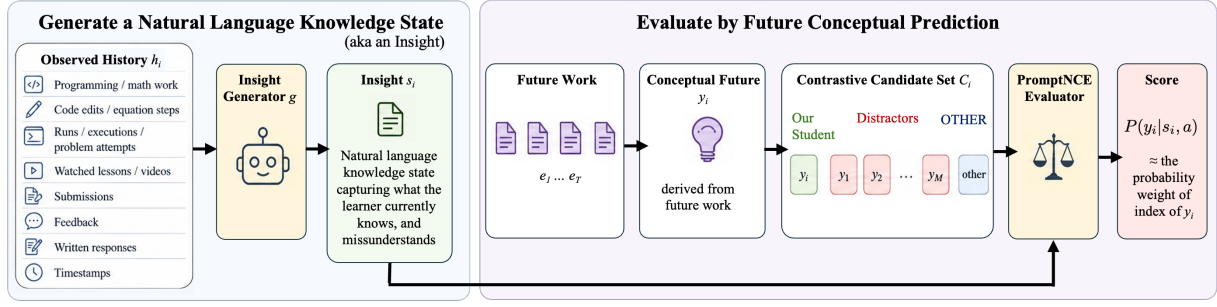


Figure 1: Overview of Learner Insight Tracing (LIT). Student histories are compressed into natural-language Insights, which are evaluated by their ability to identify the student’s future conceptual trajectory among contrastive alternatives.

edge states through prediction of future student misconceptions, including future code and conceptual reasoning. Prior work in knowledge tracing focuses on low-dimensional outcomes such as correctness, mastery, or next-step performance. Our framework instead asks whether a representation preserves information about the rich textual behaviors students will produce next. We introduce a contrastive evaluation framework that scores a knowledge state by its ability to identify a student’s future work among distractors, applying PromptNCE (Woodrow and Piech, 2026) to model open-world prediction.

2. Recurrent Tracing: We compare over a dozen approaches for generating Insights and find that recurrent updating consistently outperforms non-recurrent methods. Our best method, Recurrent Rubric-Based generation, reduces history size by more than an order of magnitude while recovering much of the predictive signal of the full history. We evaluate across 2,240 students on nine assignments spanning three courses, including one open-source dataset.

3. Deployment to 451 teachers: We produce Insights for 5,449 students in an online course and provide summaries to their 451 teachers. We explore what can make an Insight *useful* in addition to predictive.

4. Code and prompts. We open-source all code and prompts at anonymized.com.

2 Background and Related Work

Across student modeling and personalization, a central problem is how to compress a person’s interaction history into a representation that supports prediction of future behavior. We study this problem for open-ended programming and math reasoning histories, focusing on natural-language summaries as inspectable learner states whose value can be measured by their information about future code.

Knowledge tracing aims to infer a student’s latent knowledge state from a history of interactions and use that state to predict future performance. Early work modeled student knowledge as skill-level latent mastery, as in Bayesian Knowledge Tracing, while later neural approaches represented student state with recurrent hidden states, memory networks, and attention-based sequence models (Corbett and Anderson, 1994; Piech et al., 2015; Zhang et al., 2017; Pandey and Karypis, 2019; Ghosh et al., 2020; Shen et al., 2024). Recent work has begun to incorporate large language models into knowledge tracing, either by formatting student histories as text, using LLMs to augment sparse student histories, or aligning LLMs with KT-style prediction tasks (Norris et al., 2025; Wang et al., 2025; Kim et al., 2024; Scarlatos et al., 2024). We inherit the central KT goal of predicting future student behavior, but differ in the form of the student state: rather than using binary mastery, opaque hidden states, or fixed concept vectors, we represent student state as a natural-language summary of open-ended programming history.

Our setting is closest to work on open-ended knowledge tracing and programming-trace modeling, where the prediction target is richer than binary correctness. Open-ended Knowledge Tracing predicts student code directly, extending KT from correctness labels to free-form programming responses (Liu et al., 2022). TIKTOC further incorporates test-case outcomes for open-ended coding tasks, while KCGen-KT uses LLM-generated knowledge components to support interpretable KT over coding problems (Duan et al., 2024, 2025). Other work models student programming behavior through large-scale program traces, code-edit embeddings, error labels, or student-like code generation (Ross et al., 2025; Heickal and Lan, 2025; Shaka et al., 2024; Miroyan et al., 2025). These

systems generally predict code, tests, errors, edits, or discrete knowledge components directly. In contrast, our intermediate object is an inspectable text representation of the learner, and we evaluate that representation by whether it preserves information about future code.

A separate line of work studies interpretable intermediate representations. Concept bottleneck models require predictions to pass through human-interpretable concepts, and explanation bottleneck models extend this idea to textual rationales (Koh et al., 2020; Yamaguchi and Nishida, 2024). In education, interpretable KT has often used concept-level mastery slots, as in memory-based KT, or LLM-generated explanations alongside predictions (Zhang et al., 2017; Li et al., 2024). The closest recent work is Language Bottleneck Models and CIKT, which use natural language to represent student knowledge state before predicting future performance (Berthon and van der Schaar, 2025; Li et al., 2025). Our work follows this broader bottleneck view, but shifts the setting to longitudinal CS1 programming histories with future open-ended code targets. This distinction matters because programming traces contain evolving attempts, syntax errors, partial strategies, and misconceptions that are difficult to capture with a small set of fixed concepts.

We evaluate summaries using contrastive prediction: given a summary and several candidate futures, a model must identify which future belongs to the same student. This connects our evaluation to contrastive representation learning, where InfoNCE-style objectives estimate how much information one representation preserves about another variable (van den Oord et al., 2018). LLM-based contrastive prompting has recently been used to estimate pointwise mutual information from candidate selections, providing a way to evaluate natural-language representations without training a task-specific classifier (Woodrow and Piech, 2026). We use this perspective to evaluate whether a learner summary preserves information about future programming behavior.

3 Learner Insight Tracing

We pose the challenge of generating *Natural-Language Knowledge States*, which we also refer to as *Insights*, for learning environments in which students produce open-ended work such as programming assignments, mathematical reasoning, and

essays. An Insight is a compact natural-language description of a student’s inferred understanding, misconceptions, and likely future difficulties derived from observable learning behavior. Unlike latent embeddings, Insights are directly interpretable by teachers and can transfer across platforms and downstream systems without requiring access to a specific model architecture or training procedure.

For each student i , let h_i denote the observable learning history available before a target point in time. In programming environments, h_i may include code edits, submissions, execution outputs, error messages, timestamps, lesson interactions, feedback, and written responses. More generally, h_i may include any multimodal evidence of learning that can be represented as input to a language model. An Insight is generated by a function $s_i = g(h_i)$, where s_i is a natural-language summary describing the student’s inferred understanding and misconceptions based on their observed learning history.

3.1 Evaluation: Predicting Student Futures

We evaluate an Insight by its ability to predict future student behavior. This follows the central intuition underlying knowledge tracing: if a representation captures meaningful information about a student, it should help predict what that student will do next. Formally, let a denote a new assignment and let y_i denote the student’s future learning behavior on that assignment. Ideally, we would score an Insight using $P(y_i | s_i, a)$, where higher probability indicates that the Insight contains more information about the student’s future behavior.

Traditional knowledge tracing methods typically evaluate latent states by predicting correctness labels or categorical responses. More recent approaches instead attempt to predict exact future work such as student code. In our initial experiments, however, exact code prediction often rewarded superficial stylistic regularities such as formatting conventions, commenting style, or variable naming rather than conceptual understanding. We therefore evaluate against a student’s near-future conceptual trajectory rather than the exact surface form of future work.

For each student i , we observe a sequence of future work snapshots $\{e_1, \dots, e_T\}$ captured during the student’s first T minutes on a new unseen assignment. In programming environments these snapshots may correspond to edits, executions, or submissions; more generally they may correspond

to work products recorded over time. We convert this trajectory into a future conceptual summary y_i by prompting a language model to describe the understandings and misconceptions visible across the trajectory. An example of one of these prompts can be found in Appendix B.1.

Directly estimating $P(y_i | s_i, a)$ is difficult because future student behavior is open-ended and highly variable. Training a task-specific predictor would require labeled datasets pairing Insights with future outcomes and retraining separately for each educational setting. Generating futures directly from an Insight creates a second problem: evaluating similarity between generated and observed student work is fundamentally ambiguous. For example, `print(var)` and `print("var")` are highly similar under many text-based similarity metrics despite reflecting different conceptual understandings of variables and strings.

We therefore evaluate predictive utility through contrastive future identification. The key idea is that a generic Insight should assign similar probability to many students' futures because it does not contain enough student-specific information to distinguish one student from another. In contrast, a strong Insight should make the student's actual future identifiable among plausible futures produced by other students working on the same task.

3.2 Conditional Probability with PromptNCE

Our evaluation requires estimating $P(y_i | s_i, a)$, where s_i is the student's Insight, a is a new assignment, and y_i is the student's future learning behavior. Directly eliciting this probability from a language model is difficult because the space of plausible futures is effectively open-ended. We therefore use PromptNCE (Woodrow and Piech, 2026), which reframes conditional prediction as a contrastive identification task.

For a focal student i , we construct a candidate set $C_i = \{y_i\} \cup \{y_j : j \in D_i\} \cup \{\text{OTHER}\}$, containing the student's true future y_i , K distractor futures sampled from other students working on the same assignment and time window, and an explicit OTHER option representing futures outside the sampled set. The candidates are shuffled uniformly at random before evaluation. Let I denote the random variable corresponding to the index of the true future within the shuffled candidate set. The evaluator receives the Insight s_i , assignment a , and candidate set C_i , and returns a probability distribution over candidate indices $P(I = x | s_i, a, C_i)$. If

x^* denotes the index corresponding to the student's true future, we score the Insight using

$$P(y_i | s_i, a) \approx P(I = x^* | s_i, a, C_i).$$

Intuitively, an Insight receives a high score when it makes the student's actual future identifiable among plausible futures from other students solving the same assignment. Because distractors are drawn from students working on the same task and within the same time window, the evaluation controls for assignment content and instead measures whether the Insight captures student-specific predictive information.

Why does PromptNCE work? Under the PromptNCE construction, the contrastive identification probability behaves like an approximation to the open-vocabulary conditional probability $P(y_i | s_i, a)$. Suppose true futures are sampled from the conditional distribution $P(y | s_i, a)$ while distractor futures are sampled independently from the marginal distribution $P(y)$. Then the probability of selecting candidate x satisfies

$$P(I = x | s_i, a, C_i) \propto \frac{P(c_x | s_i, a)}{P(c_x)},$$

where c_x denotes the future associated with candidate x . Thus, futures receive high probability when they become especially likely given the student's Insight relative to how common they are overall. Standard contrastive estimation normalizes only over the sampled candidate set, implicitly assuming that one of the listed candidates must be correct. PromptNCE instead introduces an explicit OTHER category so the evaluator can reserve probability mass for futures outside the sampled set. This avoids the closed-world normalization used in standard contrastive estimation and makes the resulting score behave much more like the open-vocabulary conditional probability $P(y_i | s_i, a)$.

Why not use model probabilities? A natural alternative is to estimate $P(y_i | s_i, a)$ directly using token log probabilities from a language model. For example, one could prompt the model with a student's Insight s_i and assignment a , then measure the likelihood assigned to the future conceptual summary y_i . In practice, however, sequence probabilities over long natural-language generations are poorly calibrated and strongly influenced by superficial properties such as phrasing, verbosity, and lexical frequency. We tested this alternative on

$N = 20$ student traces from **Course 1**, an on-line introductory computer science course, with expert-written Insights, computing log probabilities with Meta-Llama-3.1-8B-Instruct (AI@Meta, 2024). For each student, we compared the expert Insight against a generic context-free baseline summary and asked whether the evaluation metric correctly preferred the expert version. PromptNCE selected the expert Insight 90.2% of the time, while several token-probability baselines achieved only 9.5% – 13.5% accuracy. This suggests that contrastive identification is substantially better aligned with evaluating student-specific predictive information than raw sequence likelihood.

3.3 Predicting Exact Work

Our primary target y_i is a natural-language summary of the student’s future conceptual trajectory rather than the exact code the student produces. Exact work prediction is more concrete, but in practice it often rewards stylistic regularities such as formatting or variable naming rather than conceptual understanding. We therefore evaluate conceptual summaries as the primary target and use exact future work only as a secondary metric. Using the same PromptNCE construction, we replace future summaries with raw future code snapshots, or math reasoning, π_i and score $P(\pi_i \mid s_i, a)$. This provides a stricter but substantially noisier signal of understanding. We refer to the contrastive score under the conceptual target as the **conceptual prediction score** and under the raw work target as the **exact work prediction score**.

4 Generating Insights

Each generator takes the observed history window h_i and produces a natural-language knowledge state $s_i = g(h_i)$, also known as an Insight.

Reference methods. Two methods define the range of possible predictive performance. *Full History* uses the complete observed history directly as the representation without generating a summary. This estimates the predictive signal available before any compression is applied and serves as an approximate upper bound. *Context-Free Baseline* returns the same generic insight for every student regardless of their history. This provides a lower bound for student-specific predictive information.

Prompted baselines. Our primary baseline is *Expert Written Prompt*, which prompts a language model to summarize the student’s history into an

insight describing what the student appears to understand, misunderstand, and struggle with. We test two direct modifications. *Expert Written Prompt + Assignment* adds the target assignment description to the generation prompt, testing whether knowledge of the future task helps the generator produce a more predictive insight. *Expert Written Prompt + 50-Word Cap* adds the same task context but constrains the output to fifty words, testing whether a short, committed representation improves predictive utility.

Calibration. Calibration methods ask whether restricting the insight to well-supported claims improves predictive utility. *Claim-Calibrated* instructs the model to include only claims supported by the observed history during generation. *Calibrated Rewrite* first generates an insight using the *Expert Written Prompt* and then passes it through a second prompt that rewrites the insight to remove or hedge unsupported claims based on the strength of the evidence in the observed history. This comparison separates calibration as an integrated constraint on generation from calibration as a post-processing step.

In-context learning and retrieval. Several methods test whether examples from prior students improve insight generation. *Activity kNN* retrieves prior students with similar activity histories and provides their histories and future outcomes as in-context examples for the generator. *Prior Cohort Statistics* provides aggregate statistics from prior students without individual retrieved examples, isolating the value of population-level priors. *Activity kNN + Prior Cohort Statistics* combines both.

We also test retrieval over inferred learner states rather than over raw activity. *HyDE* first drafts an initial insight using the *Expert Written Prompt*, then retrieves students from a prior course offering whose draft insights are similar in embedding space, and then revises the insight using the retrieved examples. *HyDE + Calibration* adds calibration instructions to the revision step. These methods are not strictly zero-shot since they require data from prior course offerings.

Structured output. *Rubric-Based* generates an insight by asking the model to organize its summary around three structured categories: recent progress, current misconceptions, and likely future difficulties. *Chain-of-Thought* asks the model to reason step-by-step through the history before producing a summary.

Recurrent insight generation. Most non-

Method	Course 1 $n = 900$ SEM ≤ 0.009	Course 2 $n = 578$ SEM ≤ 0.013	Course 3 $n = 762$ SEM ≤ 0.011	Overall $n = 2240$ SEM ≤ 0.006	Length chars/student -	Cost \$/student SEM ≤ 0.001
Full History	0.181	0.197	0.179	0.184	86,119	–
Recurrent + Rubric Based	0.141	0.139	0.156	0.146	4,872	0.049
Recurrent + HyDE	0.148	0.138	0.144	0.144	4,737	0.079
Rubric Based	0.127	0.124	0.162	0.138	3,519	0.028
Recurrent + Chain of Thought	0.122	0.122	0.145	0.130	3,458	0.045
Calibrated Rewrite	0.129	0.128	0.129	0.129	952	0.047
Chain of Thought	0.121	0.119	0.144	0.128	3,459	0.027
HyDE	0.123	0.124	0.133	0.127	1,619	0.052
HyDE + Calibration	0.124	0.112	0.139	0.126	1,494	0.050
Expert Written Prompt	0.127	0.116	0.128	0.124	1,052	0.024
Activity kNN Retrieval	0.109	0.115	0.110	0.111	988	0.025
25% of History Only	0.105	0.112	0.117	0.111	998	0.007
Prior Cohort Statistics	0.105	0.112	0.110	0.108	994	0.025
Context Free Baseline	0.046	0.041	0.034	0.041	42	–

Table 1: Conceptual prediction results for a representative subset of generators. Scores are mean PromptNCE probabilities assigned to the true future conceptual trajectory. Each course includes three target assignments. The best compressed method outperforms Expert Written Prompt under bootstrap resampling, with uncorrected $p = 0.001$ and corrected $p = 0.007$. SEM values are upper bounds across methods within each column.

recurrent methods generate an insight in a single pass over the full history window. This creates two practical problems. First, the window length is a hyperparameter whose optimal value may depend on the course and the data logging granularity. Second, in deployment, teachers need current summaries, but recomputing each summary from scratch is inefficient. Recurrent generators address both problems. We split the observed history into chronological chunks. At each step, the generator receives the current insight and the next chunk of student activity and produces an updated insight. We evaluate recurrent variants of *Rubric-Based*, *Chain-of-Thought*, and *HyDE*.

To test whether recurrent methods benefit from retaining information over time or simply from seeing recent activity, we include a *25% of History Only* ablation that passes only the most recent 25% of the 7-day history to generate a summary. This is the same chunk available at the final recurrent update, but without the accumulated insight from earlier steps. The full set of generators and their purposes is summarized in Appendix Table 2.

5 Experimental Setup

5.1 Datasets

We evaluate across three courses that span different subject matter, institution types, and student populations. **Course 1** is a large, online, introductory Python course designed for beginner programmers. **Course 2** is a probability course at a large R1 university with assignments involving probabilistic

modeling, inference, and mathematical reasoning. The **Course 3** is from FalconCode (de Freitas et al., 2023), an open-access CS1 dataset spanning multiple course offerings. The first two courses contain private student data and cannot be released.

Each setting contributes three target assignments, for nine course-assignment buckets and 2,240 held-out students total. We reserved 50 students (not in the 2,240) per bucket for prompt development, but no reported results use these students.

5.2 Hyperparameters and Implementation

All insight generation and evaluation use gemini-3.1-pro-preview (Google, 2026) via the Gemini API with flex-tier inference. The 7-day history window and 5-minute future window were selected based on performance on the development set. Our implementation of PromptNCE uses $K=9$ distractors per student. For recurrent methods, we split each student’s history into four chunks using event-density chunking and update the insight sequentially: $s_1 = g(\text{chunk}_1)$ and $s_i = g(s_{i-1}, \text{chunk}_i)$ for $i = 2, 3, 4$. For retrieval methods, support examples are drawn from prior course offerings, mimicking real-world deployment. The support example sets range from 99 to 3,351 students per assignment. For HyDE, we embed Insights using gemini-embedding-2 (768 dimensions, cosine similarity) and retrieve the top $K=2$ neighbors.

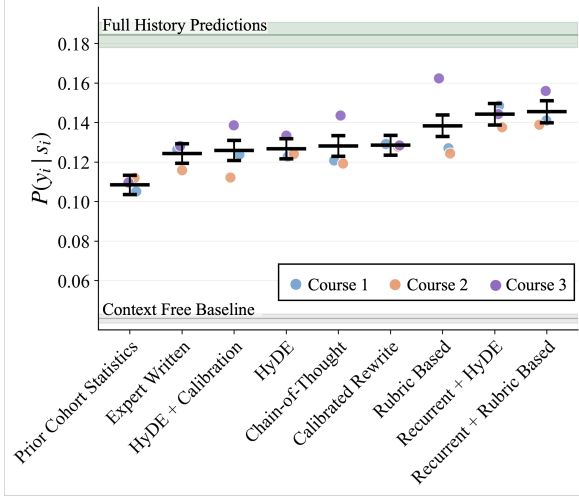


Figure 2: Conceptual prediction scores across methods and courses. Points show mean PromptNCE score, with error bars showing standard error of the mean across students. Recurrent structured methods consistently outperform single-pass baselines. Full History and the Context-Free Baseline define approximate upper and lower bounds.

6 Results

Table 1 reports average conceptual prediction scores, length, and cost for a representative subset of Insight generators. Appendix Tables 3 and 4 report the full conceptual results, and Appendix Table 5 reports exact work prediction. Figure 2 shows conceptual prediction by method and course. Figure 3 shows conceptual prediction as a function of Insight length, with a dashed reference line connecting the Context-Free Baseline to Full History in log-length space.

Full History scores highest, with a mean PromptNCE score of 0.184, while the Context-Free Baseline scores 0.041. This confirms that the observed histories contain predictive signal and gives approximate upper and lower reference points for compression.

The best compressed methods recover much of the Full History signal and reduce representation size by more than an order of magnitude. Recurrent Rubric-Based reaches 0.146 on conceptual prediction and 0.221 on exact work prediction, while reducing the representation from 86,119 characters per student to 4,872 characters at a generation cost of \$0.049 per student. Recurrent + HyDE reaches a similar conceptual prediction score (0.144) and exact work prediction score (0.216).

On conceptual prediction, recurrent updating is the most consistent advantage among compressed methods. Each recurrent method improves over its non-recurrent counterpart on conceptual prediction.

Recurrent Rubric-Based improves over Rubric-Based (0.146 vs. 0.138), Recurrent + HyDE improves over HyDE (0.144 vs. 0.127), and Recurrent Chain-of-Thought improves over Chain-of-Thought (0.130 vs. 0.128). A natural alternative explanation is that recurrent methods work because they emphasize recent activity, rather than because they preserve useful information over time. We test this with the Recent History Only ablation, which gives the model only the most recent 25% of the 7-day history. This is the same raw history chunk available at the final recurrent update, but without the accumulated insight from earlier chunks. Recent History Only scores 0.111 on conceptual prediction and 0.140 on exact work prediction, far below the recurrent methods in both cases. This suggests that recurrence is not merely benefiting from recent activity and that the updated insight carries forward earlier evidence that remains predictive.

Structured output helps independently of recurrence. Rubric-Based scores 0.138 on conceptual prediction, compared to 0.128 for Chain-of-Thought and 0.124 for the Expert Written Prompt. Rubric-Based also scores 0.219 on exact work prediction, compared to 0.165 for Chain-of-Thought and 0.149 for the Expert Written Prompt. Asking the model to organize insights around pedagogically meaningful categories appears to preserve more predictive information than an unconstrained summary. Rubric-Based is also the cheapest competitive method in Table 1, costing \$0.028 per student, compared to \$0.049 for Recurrent Rubric-Based and \$0.079 for Recurrent + HyDE. In Figure 3, Rubric-Based also lies above the dashed reference line, suggesting that its advantage is not explained only by producing a longer insight.

Calibration results are mixed. Calibrated Rewrite modestly improves over the Expert Written Prompt on conceptual prediction, 0.129 vs. 0.124, whereas Claim-Calibrated Generation drops to 0.116. This suggests calibration is more useful as post-processing than as a constraint during initial generation.

Retrieval and cohort-prior methods provide weaker gains. Activity kNN, Prior Cohort Statistics, and their combination score 0.111, 0.108, and 0.114, all below the Expert Written Prompt at 0.124. HyDE performs slightly better as a single-call method, but its main value appears when combined with recurrence, where Recurrent + HyDE reaches 0.144.

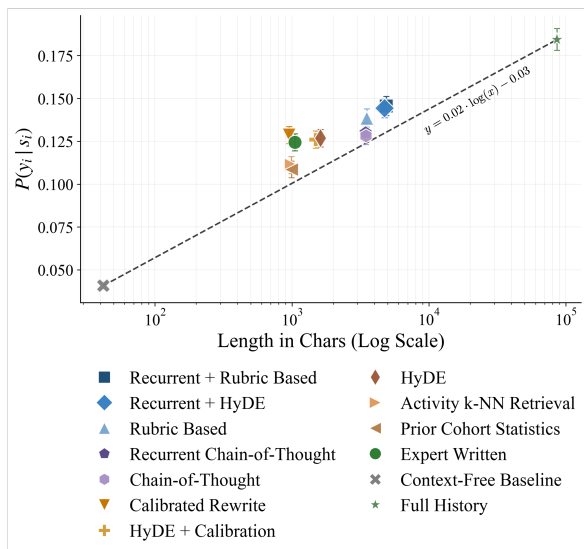


Figure 3: Conceptual prediction score as a function of representation length. Length is measured in characters per student on a log scale. Error bars are standard error of the mean. The dashed line connects the Context-Free Baseline and Full History; methods above it preserve more predictive information than this reference trend.

7 Deployment: Class Summaries from Learner Insights

We deployed Learner Insights in a large online introductory programming course. Volunteer teachers each meet weekly over Zoom with a small group of about a dozen students. Between meetings, students continue working independently, but volunteer teachers cannot observe this work directly. Each teacher received a per-student Insight for every student in their group plus a one-paragraph group summary (Appendix C), shown on their course homepage before the weekly meeting. We deployed insights to 451 volunteer teachers covering 5,449 students over two consecutive weeks.

We measured teacher response using two five-point Likert items asking whether the summary was “a useful description of my students before section” and “an accurate description of my students before section.” Debrief response rates were high, with 62% of teachers responding in the first deployment week and 53% in the second. Teachers rated the summaries above neutral on both dimensions. Usefulness ratings were 3.85 ± 0.06 (deployment week 1, $n=280$) and 3.77 ± 0.06 (deployment week 2, $n=238$). Accuracy ratings were 3.60 ± 0.05 (deployment week 1, $n=278$) and 3.69 ± 0.06 (deployment week 2, $n=236$).

To understand what deployed insights look like in practice, we sampled six group summaries

from each of four rating categories: highly-useful, poorly-useful, highly-accurate, and poorly-accurate. We coded each holistic summary on specificity, actionability, accuracy-claim hedging, tone, and completeness. Summaries were broadly consistent in structure across rating bands. All twenty-four named specific Python concepts such as variables, type conversion, f-strings, and while-loop state. High-rated and low-rated summaries contained similar kinds of information, suggesting that this type of structural variation did not predict teacher ratings.

What stands out most is that insights surfaced behaviors volunteer teachers had no way to observe between meetings. Group summaries contained phrases like “variables update themselves automatically like a spreadsheet,” tracing back to a per-student note that the student “expect[s] variables to continuously update like spreadsheet formulas.” Another described a student who “mistakenly tried to translate built-in Python keywords into Spanish” (*importar*, *desde*). Some summaries flagged students who pasted complete solutions they found when stuck. Others tied observations directly to upcoming material. For example, one noted “[a]s you prepare for the [anonymized] problem, it will be helpful to know that several students still confuse plain text with variable names.” These are the kinds of signals that are invisible without access to intermediate student work and insights about students’ progress generated in real time.

8 Conclusion

We introduced Learner Insight Tracing, a framework for generating natural-language knowledge states and evaluating them by whether they predict future student work. Across three courses, recurrent structured Insights preserved substantial predictive signal while compressing student histories by more than an order of magnitude, and deployment results suggest that these summaries can surface information teachers do not otherwise observe. Future work should test whether predictive utility translates into better teacher decisions and student outcomes, evaluate longer prediction horizons, and validate the approach across additional models, courses, and learning environments.

9 Limitations

All insight generation and evaluation use a single frontier model (Gemini 3.1 Pro Preview). Results

may not generalize to other frontier or open-source models. All courses and student histories are in English. Two of the three datasets contain private student data and cannot be released, limiting reproducibility to the FalconCode course. Retrieval-based methods (HyDE, Activity kNN) require data from a prior course offering and are not fully zero-shot, though they do not require training a model. Our evaluation measures predictive utility at a fixed 5-minute horizon; whether insights that score well here also predict student behavior over longer time horizons is an open question.

A high conceptual prediction score indicates that an insight captures student-specific information, but does not guarantee that the insight is accurate, actionable, or useful to a teacher in practice. Our deployment provides initial evidence that teachers find the summaries useful, but it was conducted in a single course and the relationship between predictive utility and teacher-perceived quality remains an open question.

References

AI@Meta. 2024. [Llama 3 model card](#). Large language model model card.

Antonin Berthon and Mihaela van der Schaar. 2025. [Language bottleneck models for qualitative knowledge state modeling](#). *Preprint*, arXiv:2506.16982. Current canonical arXiv title differs from the earlier planning title.

Albert T. Corbett and John R. Anderson. 1994. [Knowledge tracing: Modeling the acquisition of procedural knowledge](#). *User Modeling and User-Adapted Interaction*, 4:253–278.

Adrian de Freitas, Joel Coffman, Michelle de Freitas, Justin Wilson, and Troy Weingart. 2023. [Falconcode: A multiyear dataset of python code samples from an introductory computer science course](#). In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, SIGCSE 2023, page 938–944, New York, NY, USA. Association for Computing Machinery.

Zhangqi Duan, Nigel Fernandez, Alexander Hicks, and Andrew Lan. 2024. [Test case-informed knowledge tracing for open-ended coding tasks](#). *Preprint*, arXiv:2410.10829.

Zhangqi Duan, Nigel Fernandez, Arun Balajiee Lekshmi Narayanan, Mohammad Hassany, Rafaella Sampaio de Alencar, Peter Brusilovsky, Bitu Akram, and Andrew Lan. 2025. [Automated knowledge component generation for interpretable knowledge tracing in coding problems](#). *Preprint*, arXiv:2502.18632.

Aritra Ghosh, Neil Heffernan, and Andrew S. Lan. 2020. [Context-aware attentive knowledge tracing](#). In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2330–2339.

Google. 2026. Gemini 3.1 pro preview. <https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview>. Large language model.

Hasnain Heickal and Andrew Lan. 2025. [Learning code-edit embedding to model student debugging behavior](#). *Preprint*, arXiv:2502.19407.

JongWoo Kim, SeongYeub Chu, Bryan Wong, and Mun Yi. 2024. [Not all options are created equal: Textual option weighting for token-efficient llm-based knowledge tracing](#). *Preprint*, arXiv:2410.12872.

Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. 2020. [Concept bottleneck models](#). In *Proceedings of the 37th International Conference on Machine Learning*, pages 5338–5348.

Haoxuan Li, Jifan Yu, Yuanxin Ouyang, Zhuang Liu, Wenge Rong, Juanzi Li, and Zhang Xiong. 2024. [Explainable few-shot knowledge tracing](#). *Preprint*, arXiv:2405.14391.

Runze Li, Siyu Wu, Jun Wang, and Wei Zhang. 2025. [CIKT: A collaborative and iterative knowledge tracing framework with large language models](#). *Preprint*, arXiv:2505.17705.

Naiming Liu, Zichao Wang, Richard Baraniuk, and Andrew Lan. 2022. [Open-ended knowledge tracing for computer science education](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3849–3862, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Mihran Miroyan, Rose Niousha, Joseph E. Gonzalez, Gireeja Ranade, and Narges Norouzi. 2025. [Parastudent: Generating and evaluating realistic student code by teaching llms to struggle](#). *Preprint*, arXiv:2507.12674.

Max Norris, Kobi Gal, and Sahan Bulathwela. 2025. [Next token knowledge tracing: Exploiting pretrained llm representations to decode student behaviour](#). *Preprint*, arXiv:2511.02599.

Shalini Pandey and George Karypis. 2019. [A self-attentive model for knowledge tracing](#). In *Proceedings of the 12th International Conference on Educational Data Mining*.

Chris Piech, Jonathan Spencer, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. 2015. [Deep knowledge tracing](#). *Preprint*, arXiv:1506.05908.

Alexis Ross, Megha Srivastava, Jeremiah Blanchard, and Jacob Andreas. 2025. [Modeling student learning with 3.8 million program traces](#). *Preprint*, arXiv:2510.05056.

Alexander Scarlatos, Ryan S. Baker, and Andrew Lan. 2024. [Exploring knowledge tracing in tutor-student dialogues using llms](#). *Preprint*, arXiv:2409.16490.

Martha Shaka, Diego Carraro, and Kenneth Brown. 2024. [Error tracing in programming: A path to personalised feedback](#). In *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*, pages 330–342, Mexico City, Mexico. Association for Computational Linguistics.

Shuanghong Shen, Qi Liu, Zhenya Huang, Yonghe Zheng, Minghao Yin, Minjuan Wang, and Enhong Chen. 2024. [A survey of knowledge tracing: Models, variants, and applications](#). *IEEE Transactions on Learning Technologies*, 17:1898–1919.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. [Representation learning with contrastive predictive coding](#). In *NeurIPS Workshop on Relational Representation Learning*.

Ziwei Wang, Jie Zhou, Qin Chen, Min Zhang, Bo Jiang, Aimin Zhou, Qinchun Bai, and Liang He. 2025. [LLM-KT: Aligning large language models with knowledge tracing using a plug-and-play instruction](#). *Preprint*, arXiv:2502.02945.

Juliette Woodrow and Chris Piech. 2026. [PromptNCE: Pointwise mutual information predictions using only llms and contrastive estimation prompts](#). *Preprint*, arXiv:2605.21776.

Shin’ya Yamaguchi and Kosuke Nishida. 2024. [Toward explanation bottleneck models](#). In *MINT Workshop at NeurIPS 2024*.

Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. 2017. [Dynamic key-value memory networks for knowledge tracing](#). In *Proceedings of the 26th International Conference on World Wide Web*, pages 765–774.

A Disclosure on LLM Usage

LLMs were used to improve and polish writing for clarity and conciseness. We also used LLMs to generate LaTeX code for tables and help us with generating our main overview figure. No LLMs were used to create any of our results figures. We used LLMs to learn about and understand the methods that inspired this work. We also sometimes used LLMs to help us better understand the methods as we read through the papers. We used LLMs to streamline the coding process, but everything was checked by the authors at every step. All experiments and analyses were run by the authors.

The authors have thoroughly reviewed all LLM-generated content and take full responsibility for everything in this work.

B Prompt Excerpts

We include abbreviated prompt excerpts to document the core evaluation and generation instructions. The full prompt text, including course-specific assignment descriptions, formatting wrappers, parsing code, and exact templates used in experiments, will be released with the code.

B.1 Conceptual Future Summary Prompt

This prompt converts a student’s first {window_minutes} minutes of work on the target assignment into the conceptual future summary y_i , which is used as the primary target in conceptual prediction.

```
You are an expert computer science teacher. You will be
given chronological snapshots showing one student's
intermediate work during their first {
window_minutes} minutes on a problem.

Describe the student at a high level: what misconceptions
they have, what they understand clearly, and
anything they resolved during the window. Focus on
conceptual understanding as revealed by behavior in
the snapshots, not surface syntax or variable
names. Only include claims directly supported by
the snapshots.

Return ONLY a JSON object with exactly these four keys:
{
  "overview": string,
  "misconceptions": [{"claim": string, "evidence": string
}, ...],
  "clear_understandings": [{"claim": string, "evidence":
string}, ...],
  "resolved_during_window": [{"claim": string, "evidence":
string}, ...]
}

Hard constraints:
- Arrays can be empty. Do not pad.
- Do not assert anything without direct evidence.
- Do not quote variable names, function names, code
tokens, syntax, indentation, formatting, or casing.
- Write in plain teacher language.

ASSIGNMENT:
{assignment_brief}

SNAPSHOTS:
{snapshots_block}
```

B.2 Conceptual Prediction Scoring Prompt

This prompt scores an insight by asking whether it identifies the student’s future conceptual trajectory among shuffled candidate futures from students working on the same assignment.

You are an expert computer science teacher. You will be shown a short summary of one student's prior understanding, misconceptions, habits, and strategies.

You will then see several candidate concept-state accounts. Each candidate is a structured account of one real student's first {time_horizon} minutes of work on a new assignment. Each account contains: overview, misconceptions, clear understandings, and resolved during the window.

TASK:

Return a probability mass function estimating how likely each candidate is to be the future work account of the student described in the summary. Include "OTHER" for the probability that the student's actual future concept state is not represented by any listed candidate.

Calibration guidance:

- The candidate list is partial, not exhaustive.
- Use OTHER when the summary is not specific enough to choose confidently, or when none of the listed candidates fit.

Output ONLY JSON:

```
{
  "reasoning": "3 to 5 sentences referencing candidates by letter",
  "pmf": {"A": number, "B": number, ..., "OTHER": number}
}
```

The PMF must include every candidate plus OTHER, and probabilities must sum to 1.

PROBLEM:

```
{assignment_brief}
```

SUMMARY:

```
{summary}
```

CANDIDATES:

```
{candidate_block}
```

C Insight Generators

Table 2 summarizes the insight generators evaluated in our experiments. Each generator takes the same observed history window and produces a natural-language insight; the comparison isolates different design choices, including task context, concision, calibration, retrieval, structured output, recurrent updating, and history length.

Appendix: Deployment Details

Summary surfaces. Summaries appeared in three places in the teacher's pre-meeting workflow: the course homepage, the group page, and a per-student detail page. The post-meeting debrief form re-surfaced the group summary alongside the rating items.

Freshness. Each insight was generated within 24 hours before the weekly meeting. Because students could continue working after generation, the per-student detail page also displayed up-to-date completion statistics read freshly at page load, so teachers could see whether a student had made additional progress since the summary was written.

Inactive students. For students with insufficient recent activity, we bypassed the LLM and displayed a fixed message: "Not enough progress over past 7 days for meaningful AI summary."

Qualitative coding. We sampled six group summaries from each of four rating categories — highly useful (≥ 4), poorly useful (≤ 2), highly accurate (≥ 4), and poorly accurate (≤ 2) — for 24 summaries total. We coded each on five dimensions: specificity (does it name concrete concepts or behaviors), actionability (could a teacher act on it), accuracy-claim hedging (does it scope claims appropriately), tone, and completeness.

C.1 Holistic Synthesis Prompt

The prompt used to synthesize a section-level holistic summary from the per-student Insights is reproduced below. At call time, the placeholder {weekly_course_context} is replaced with a rendered prose block describing the course week, and {student_summaries} is replaced with the concatenated per-student Insights in the section. For the anonymous review version, institution and course names have been redacted.

You are an expert computer science teacher at [anonymized institution] with decades of experience teaching introductory programming. Below are text-based-knowledge-states describing what each student knows and doesn't know based on their work in the past 7 days in the course. Combine them into one 2-3 sentence summary of the entire section that a teacher will read when preparing to teach their next discussion section. In the summary, you should include what would be helpful for the teacher to know before teaching this group. But you should not try to tell them what to teach or how to teach. We don't want to infringe on their teaching creativity, we just want to describe their students and provide information.

One thing to note is that many of the teachers are not native english speakers. So you need to write your summary in very clear and direct language. Don't use complicated technical terminology. And just describe the students, perhaps grouping them based on what they know / don't know etc.

Here is some context on the course -

```
\texttt{\{weekly\_course\_context\}}
```

```
\texttt{\{student\_summaries\}}
```

Generator	Key idea	Purpose in comparison
Full History	Uses the full observed history h_i directly as the representation, without generating a summary.	Estimates the predictive signal available before compression.
Context-Free Baseline	Returns the same generic insight for every student.	Provides a lower bound for student-specific predictive information.
Expert Written Prompt	Generates an insight from the observed history using a hand-written teacher-style prompt.	Serves as the primary prompted LLM baseline.
Expert Written Prompt + Assignment	Adds the target assignment description to the generation prompt.	Tests whether knowing the future task improves predictive utility.
Expert Written Prompt + 50-Word Cap	Adds task context and constrains the output to a short summary.	Tests whether aggressive concision improves predictive utility.
Claim-Calibrated Generation	Prompts the model to include only claims supported by the observed history during initial generation.	Tests whether evidential calibration helps when integrated into the generation prompt.
Calibrated Rewrite	First generates an insight using the Expert Written Prompt, then rewrites it with calibration instructions.	Tests whether calibration is more effective as a post-processing step.
Activity kNN Retrieval	Retrieves prior students with similar activity histories and includes their histories and future outcomes as in-context examples.	Tests whether nearest-neighbor examples based on raw activity improve generation.
Prior Cohort Statistics	Provides aggregate statistics from prior students without individual retrieved examples.	Isolates the value of population-level priors.
Activity kNN + Prior Cohort Statistics	Combines activity-based nearest-neighbor examples with aggregate cohort statistics.	Tests whether population priors add information beyond retrieved examples.
HyDE	Drafts an initial insight, retrieves similar prior students in insight-embedding space, then revises the insight using the retrieved examples.	Tests whether retrieval over inferred learner states improves generation.
HyDE + Calibration	Combines HyDE retrieval with calibration instructions during the revision step.	Tests whether summary-space retrieval and calibration provide complementary gains.
Rubric Based	Generates an insight in one call using a structured format organized around pedagogically meaningful categories.	Tests whether structured output improves predictive utility while holding generation to a single pass.
Chain of Thought	Asks the model to reason through the history before producing an insight.	Tests whether explicit reasoning improves single-pass insight generation.
Recurrent + Rubric Based	Processes the history in chronological chunks, updating a structured insight at each step.	Tests whether recurrent updating preserves useful information from long histories.
Recurrent + Chain of Thought	Processes the history recurrently while using a chain-of-thought-style generation format.	Tests whether recurrence helps when paired with explicit reasoning.
Recurrent + HyDE	Applies HyDE-style retrieval during recurrent insight generation.	Tests whether retrieval is more useful when integrated into an evolving learner state.
Recurrent + Initial HyDE	Uses HyDE retrieval only to seed the first recurrent insight state.	Tests whether retrieval helps primarily by improving the initial recurrent state.
25% of Recent History	Generates an insight from only the most recent 25% of the 7-day history.	Tests whether recurrent methods help because they preserve earlier information, rather than only because they emphasize recent activity.

Table 2: Insight generators compared in our experiments. The methods instantiate different hypotheses about what improves insight quality, including task context, concision, calibration, retrieval, structured output, recurrent updating, and history length.

Method	Course 1	Course 2	Course 3 (Public)	Overall
Full History	0.181 $\pm$ 0.009	0.197 $\pm$ 0.013	0.179 $\pm$ 0.011	0.184 $\pm$ 0.006
Recurrent + Rubric Based	0.141 $\pm$ 0.008	0.139 $\pm$ 0.011	0.156 $\pm$ 0.010	0.146 $\pm$ 0.006
Recurrent + HyDE	0.148 $\pm$ 0.009	0.138 $\pm$ 0.011	0.144 $\pm$ 0.010	0.144 $\pm$ 0.006
Recurrent + Initial HyDE	0.146 $\pm$ 0.009	0.134 $\pm$ 0.011	0.149 $\pm$ 0.010	0.144 $\pm$ 0.006
Rubric Based	0.127 $\pm$ 0.008	0.124 $\pm$ 0.010	0.162 $\pm$ 0.011	0.138 $\pm$ 0.005
Recurrent + Chain of Thought	0.122 $\pm$ 0.008	0.122 $\pm$ 0.010	0.145 $\pm$ 0.010	0.130 $\pm$ 0.005
Calibrated Rewrite	0.129 $\pm$ 0.008	0.128 $\pm$ 0.010	0.129 $\pm$ 0.009	0.129 $\pm$ 0.005
Chain of Thought	0.121 $\pm$ 0.008	0.119 $\pm$ 0.010	0.144 $\pm$ 0.010	0.128 $\pm$ 0.005
HyDE	0.123 $\pm$ 0.008	0.124 $\pm$ 0.010	0.133 $\pm$ 0.009	0.127 $\pm$ 0.005
HyDE + Calibration	0.124 $\pm$ 0.008	0.112 $\pm$ 0.009	0.139 $\pm$ 0.009	0.126 $\pm$ 0.005
Expert Written Prompt	0.127 $\pm$ 0.008	0.116 $\pm$ 0.009	0.128 $\pm$ 0.009	0.124 $\pm$ 0.005
Claim-Calibrated Generation	0.120 $\pm$ 0.008	0.111 $\pm$ 0.009	0.116 $\pm$ 0.008	0.116 $\pm$ 0.005
Activity kNN + Prior Cohort Statistics	0.112 $\pm$ 0.008	0.118 $\pm$ 0.010	0.114 $\pm$ 0.009	0.114 $\pm$ 0.005
Activity kNN Retrieval	0.109 $\pm$ 0.008	0.115 $\pm$ 0.010	0.110 $\pm$ 0.009	0.111 $\pm$ 0.005
25% of History Only	0.105 $\pm$ 0.007	0.112 $\pm$ 0.009	0.117 $\pm$ 0.009	0.111 $\pm$ 0.005
Expert Written Prompt + Assignment	0.104 $\pm$ 0.007	0.114 $\pm$ 0.010	0.115 $\pm$ 0.009	0.110 $\pm$ 0.005
Prior Cohort Statistics	0.105 $\pm$ 0.007	0.112 $\pm$ 0.010	0.110 $\pm$ 0.009	0.108 $\pm$ 0.005
Expert Written Prompt + 50-Word Cap	0.074 $\pm$ 0.007	0.103 $\pm$ 0.009	0.101 $\pm$ 0.008	0.091 $\pm$ 0.004
Context Free Baseline	0.046 $\pm$ 0.004	0.041 $\pm$ 0.005	0.034 $\pm$ 0.004	0.041 $\pm$ 0.002

Table 3: Full conceptual prediction results for all methods. Values are mean $\pm$ SEM across students. Each course includes three target assignments.

Method	Length	Cost
Full History	86,119 $\pm$ 2,060	–
Recurrent + Rubric Based	4,872 $\pm$ 33	0.049 $\pm$ 0.001
Recurrent + HyDE	4,737 $\pm$ 28	0.079 $\pm$ 0.001
Recurrent + Initial HyDE	4,800 $\pm$ 29	0.074 $\pm$ 0.001
Rubric Based	3,519 $\pm$ 19	0.028 $\pm$ 0.001
Recurrent + Chain of Thought	3,458 $\pm$ 20	0.045 $\pm$ 0.001
Calibrated Rewrite	952 $\pm$ 2	0.047 $\pm$ 0.001
Chain of Thought	3,459 $\pm$ 18	0.027 $\pm$ 0.001
HyDE	1,619 $\pm$ 5	0.052 $\pm$ 0.001
HyDE + Calibration	1,494 $\pm$ 5	0.050 $\pm$ 0.001
Expert Written Prompt	1,052 $\pm$ 2	0.024 $\pm$ 0.001
Claim-Calibrated Generation	987 $\pm$ 2	0.024 $\pm$ 0.001
Activity kNN + Prior Cohort Statistics	992 $\pm$ 1	0.025 $\pm$ 0.001
Activity kNN Retrieval	988 $\pm$ 1	0.025 $\pm$ 0.001
25% of History Only	998 $\pm$ 2	0.007 $\pm$ 0.000
Expert Written Prompt + Assignment	1,058 $\pm$ 2	0.024 $\pm$ 0.001
Prior Cohort Statistics	994 $\pm$ 1	0.025 $\pm$ 0.001
Expert Written Prompt + 50-Word Cap	313 $\pm$ 1	0.023 $\pm$ 0.001
Context Free Baseline	42 $\pm$ 0	–

Table 4: Representation length and generation cost for each method. Length is measured in characters per student, and cost is the average Gemini 3.1 Pro Preview generation cost per student. Values are mean $\pm$ SEM across students; Full History and Context-Free Baseline have no generation cost.

Method	Course 1	Course 2	Course 3	Overall	Length (chars)
Full History	0.329 ± 0.013	0.479 ± 0.018	0.716 ± 0.014	0.499 ± 0.009	$86,119 \pm 2,060$
Recurrent + Initial HyDE	0.185 ± 0.010	0.180 ± 0.013	0.294 ± 0.014	0.221 ± 0.007	$4,800 \pm 29$
Recurrent + Rubric Based	0.178 ± 0.010	0.189 ± 0.014	0.296 ± 0.014	0.221 ± 0.007	$4,872 \pm 33$
Rubric Based	0.177 ± 0.010	0.172 ± 0.013	0.304 ± 0.014	0.219 ± 0.007	$3,519 \pm 19$
Recurrent + HyDE	0.182 ± 0.010	0.187 ± 0.014	0.279 ± 0.014	0.216 ± 0.007	$4,737 \pm 28$
Recurrent + Chain of Thought	0.147 ± 0.010	0.145 ± 0.012	0.219 ± 0.013	0.171 ± 0.007	$3,458 \pm 20$
Chain of Thought	0.131 ± 0.009	0.141 ± 0.012	0.222 ± 0.012	0.165 ± 0.006	$3,459 \pm 18$
HyDE	0.150 ± 0.009	0.139 ± 0.011	0.166 ± 0.011	0.153 ± 0.006	$1,619 \pm 5$
Calibrated Rewrite	0.151 ± 0.009	0.138 ± 0.011	0.157 ± 0.010	0.150 ± 0.006	952 ± 2
Expert Written Prompt	0.152 ± 0.009	0.127 ± 0.011	0.162 ± 0.011	0.149 ± 0.006	$1,052 \pm 2$
HyDE + Calibration	0.146 ± 0.009	0.130 ± 0.011	0.166 ± 0.011	0.149 ± 0.006	$1,494 \pm 5$
Activity kNN Retrieval	0.142 ± 0.009	0.130 ± 0.011	0.170 ± 0.011	0.148 ± 0.006	988 ± 1
Activity kNN + Prior Cohort Statistics	0.136 ± 0.009	0.138 ± 0.012	0.164 ± 0.011	0.146 ± 0.006	992 ± 1
Prior Cohort Statistics	0.143 ± 0.009	0.130 ± 0.011	0.158 ± 0.011	0.145 ± 0.006	994 ± 1
Expert Written Prompt + Assignment	0.123 ± 0.009	0.133 ± 0.011	0.170 ± 0.011	0.142 ± 0.006	$1,058 \pm 2$
Claim-Calibrated Generation	0.149 ± 0.009	0.129 ± 0.011	0.139 ± 0.010	0.140 ± 0.006	987 ± 2
25% of History Only	0.132 ± 0.009	0.129 ± 0.011	0.158 ± 0.011	0.140 ± 0.006	998 ± 2
Expert Written Prompt + 50-Word Cap	0.090 ± 0.008	0.110 ± 0.011	0.119 ± 0.009	0.105 ± 0.005	313 ± 1
Context-Free Baseline	0.083 ± 0.005	0.109 ± 0.009	0.086 ± 0.006	0.090 ± 0.004	42 ± 0

Table 5: Exact-work prediction results for all methods. Scores are mean PromptNCE probabilities assigned to the true future work trajectory, reported as mean $\pm$ SEM across students. Length is measured in characters per student.